

### 3.1 Introduction

The main purpose of data compression is removing the redundancy present in data in order to minimize storage space or transmission time.

**Example 3.1.** Consider a fair dice toss. It can be modeled as a random variable with alphabet  $\{1, 2, 3, 4, 5, 6\}$  and probability distribution  $\{\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}\}$ . We can use the direct binary representation and obtain

$$1 \mapsto 001 \quad 2 \mapsto 010 \quad 3 \mapsto 011 \quad 4 \mapsto 100 \quad 5 \mapsto 101 \quad 6 \mapsto 110. \quad (3.1)$$

There are alternatives to this, though. For example,

$$1 \mapsto 00 \quad 2 \mapsto 01 \quad 3 \mapsto 100 \quad 4 \mapsto 101 \quad 5 \mapsto 110 \quad 6 \mapsto 111 \quad (3.2)$$

or even

$$1 \mapsto 1100 \quad 2 \mapsto 1010 \quad 3 \mapsto 0110 \quad 4 \mapsto 1001 \quad 5 \mapsto 0101 \quad 6 \mapsto 0011. \quad (3.3)$$

The above are all one-to-one (bijective) mappings, i.e., they are lossless, meaning that upon observing the binary string, the original message can always be recovered. For example, if the result several dice tosses is encoded using the source code in (3.2) as 1000100, we know that it corresponds to the source sequence (3, 2, 1). The same sequence encoded using the source code in (3.3) would be 011010101100, much longer.

In general, we will measure the efficiency of a code using the expected amount of bits required to represent a source. For a source taking values in  $\{1, \dots, r\}$  with probability distribution  $\{p_1, \dots, p_r\}$ , the **average length** of the code is given by

$$L_{av} = \sum_{j=1}^r p_j \ell_j \quad (3.4)$$

where  $\ell_j$  is the length (measured in binary digits) of the binary string associated with symbol  $j$ .

**Example 3.2.** According to the source code (3.1) in Example 3.1, all symbols in the source are mapped into binary strings of length 3. Therefore,  $\ell_j = 3$  for all  $j \in \{1, \dots, 6\}$  in the alphabet and the average length is simply  $L_{av} = 3$  bits per symbol. Similarly,  $L_{av} = 4$  bits per symbol for the code in (3.3). The code in (3.2) has variable lengths and  $\ell_j = 2$  for  $j \in \{1, 2\}$  and  $\ell_j = 3$  for  $j \in \{3, 4, 5, 6\}$ . As a result,  $L_{av} = \frac{8}{3} \approx 2.667$  bits per symbol. The code (3.2) clearly provides a more efficient (shorter) way of representing the source.

**Example 3.3.** You would like to set up your own telephone system that connects you with your 3 best friends using binary codes. The phone number will be a **codeword** and the set of codewords is the **code**, or **codebook**. You know that you will call Bob 50% of the times and that you will call Alice and Carol 25% of the times each. The source can be modeled using the alphabet  $\{\text{Alice}, \text{Bob}, \text{Carol}\}$  with probability distribution  $\{\frac{1}{4}, \frac{1}{2}, \frac{1}{4}\}$ . We now examine several methods to produce the phone numbers reported in Table 1 below. In code (a), Alice

**Table 1:** Phone numbers for Example 3.3.

| Friend | Probability | (a)  | (b)    | (c) | (d) | (e) | (f) |
|--------|-------------|------|--------|-----|-----|-----|-----|
| Alice  | 0.25        | 0011 | 001101 | 0   | 00  | 0   | 10  |
| Bob    | 0.5         | 0011 | 001110 | 1   | 11  | 11  | 0   |
| Carol  | 0.25        | 1100 | 110000 | 10  | 10  | 10  | 11  |

and Bob have the same number, therefore the system will not connect properly. The code (b) will work, since each has a different phone number, but the numbers are very long and the system becomes inefficient. Code (c) is a shorter code, however observe that the system will not work: when we dial 10 it is unclear whether it

means “Carol” or “Bob, Alice”. The code (c) is not **uniquely decodable**. Code (d) is the first efficient code that works and is uniquely decodable, but why are we assigning 00 to Alice if we dial 0 it is clear we wish to dial Alice? An improvement is code (e); it is uniquely decodable, but it is not the most efficient since the code is assigning a 2-bit string to the most probable symbol! Finally, (f) is the **optimal code** with an average length of  $L_{av} = 1.5$  bits per symbol. It is interesting to note that once you finish dialing, the system immediately knows that you have finished dialing. This is because no codeword is a prefix of any other, i.e., the code is **prefix-free**.

### 3.2 Prefix-free codes

Prefix-free codes are variable-length codes that guarantee unique decodability and, properly designed, are efficient in terms of average length, i.e., minimize (3.4).

Consider a discrete memoryless source  $U$  taking values over the alphabet  $\{1, \dots, r\}$ . A binary source code is the set codewords  $\mathcal{C} = \{\mathcal{C}(1), \mathcal{C}(2), \dots, \mathcal{C}(r)\}$ , where each  $\mathcal{C}(j)$  is the binary sequence used to represent the source symbol  $j \in \{1, \dots, r\}$ . For example, the code (f) in Table 1 for the source {Alice, Bob, Carol} is given  $\mathcal{C}(\text{Alice}) = 10$ ,  $\mathcal{C}(\text{Bob}) = 0$  and  $\mathcal{C}(\text{Carol}) = 11$ , or simply the set  $\{10, 0, 11\}$ . A source sequence of length  $n$  is the sequence  $(u_1, \dots, u_n)$  where each  $u_i \in \{1, \dots, r\}$ . A symbol-by-symbol encoding consists of  $\mathcal{C}(u_1) \dots \mathcal{C}(u_n)$  the concatenation of the codewords. For example, the source sequence (Alice, Alice, Carol, Bob, Carol) of length  $n = 5$  is encoded as 101011011, a binary sequence of length 9 bits.

A code is called **uniquely decodable** if no pair of different sequences  $(u_1, \dots, u_n) \neq (u'_1, \dots, u'_m)$  exist such that  $\mathcal{C}(u_1) \dots \mathcal{C}(u_n) = \mathcal{C}(u'_1) \dots \mathcal{C}(u'_m)$ , that is, for a code to be uniquely decodable, two distinct source sequences cannot be encoded into the same codeword. Recall the phone system (c) from Example 3.3. A code is called **prefix-free** if no codeword is a prefix of any other codeword.

**Theorem 3.4.** Prefix-free codes are uniquely decodable.

**Problem 3.5.** Consider the codes  $\mathcal{C}_1 = \{0, 10, 110, 111\}$  and  $\mathcal{C}_2 = \{0, 01, 011, 111\}$  for a source  $\{a, b, c, d\}$ . Are the codes uniquely decodable and/or prefix-free? Decode the sequence 001101110 using both codes.

*Solution:* The code  $\mathcal{C}_1$  is prefix-free because no codeword is a prefix of any other, and therefore the code is uniquely decodable. The code  $\mathcal{C}_2$  is not prefix-free because  $\mathcal{C}(a)$  is prefix of  $\mathcal{C}(b)$  and  $\mathcal{C}(c)$ , while  $\mathcal{C}(b)$  is also prefix of  $\mathcal{C}(c)$ . Observe though that the code is uniquely decodable because the bit 0 is like a comma when read from right to left. Using the first code, the sequence to be decoded can be uniquely grouped as 0, 0, 110, 111, 10. Since  $\mathcal{C}^{-1}(0) = a$ ,  $\mathcal{C}^{-1}(110) = c$ ,  $\mathcal{C}^{-1}(111) = d$  and  $\mathcal{C}^{-1}(10) = b$ , we decode the sequence as  $(a, a, c, d, b)$ . For the second code, we need to read from right to left as 0, 011, 01, 111, 0 to obtain  $(a, c, b, d, a)$  instead.

Any binary code can be represented as a binary tree (see Figure 3.1). Take any codeword and regard each bit as a decision as to whether to go up (0) or down (1). Thus, every codeword can be represented by a particular path in the tree, as the two examples in Figure 3.1. In the trees, the root node is denoted with a square, intermediate nodes with small circles and leaf nodes as gray circles. Edges are represented with a solid line. Labels on the right denote binary sequences. Similarly, in Figure 3.2, we illustrate the binary tree representations of the codes (c), (d), (e) and (f) from Example 3.3. The white circle node indicates a leaf with no codeword. Observe that code (c) is not prefix-free and has a codeword along the path to another codeword. Thus, a code is prefix-free if in its tree representation all codewords appear as leaves in the tree.

The following is an important result relating the existence of prefix-free codes and binary trees.

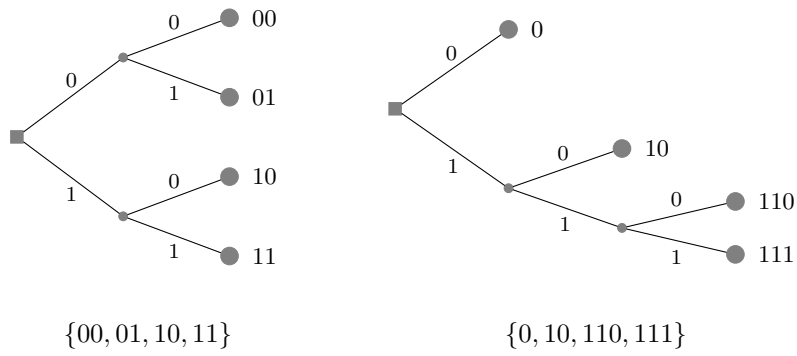
**Theorem 3.6 (Kraft's inequality).** Consider a binary source code  $\mathcal{C}$  for a source with alphabet  $\{1, \dots, r\}$  and probability distribution  $\{p_1, \dots, p_r\}$ . Then, if the code is prefix-free, then

$$\sum_{j=1}^r 2^{-\ell_j} \leq 1. \quad (3.5)$$

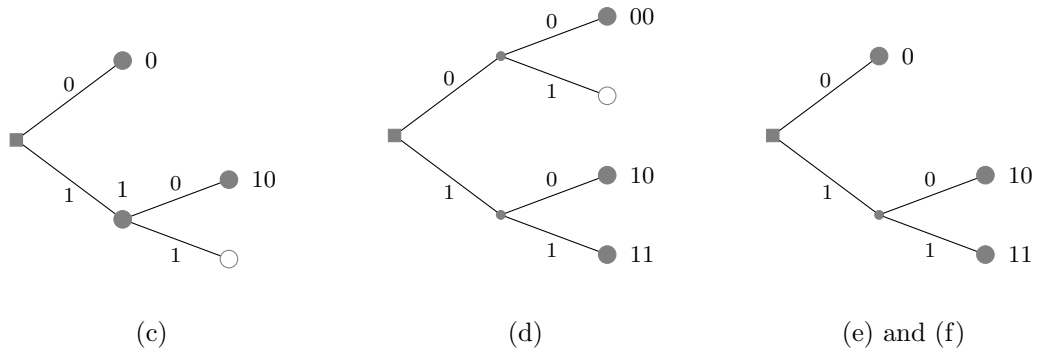
In addition, there exists a prefix-free code with lengths  $\ell_j$  for  $j = 1, \dots, r$  satisfying (3.5).

**Example 3.7.** A code with length distribution  $\{\ell_1, \dots, \ell_r\} = \{3, 4, 4, 4, 4\}$  satisfies Kraft's inequality since

$$\sum_{j=1}^r 2^{-\ell_j} = 2^{-3} + 4 \cdot 2^{-4} = \frac{1}{8} + \frac{4}{16} = \frac{3}{8} \leq 1. \quad (3.6)$$



**Figure 3.1:** Binary tree representation of two binary source codes.

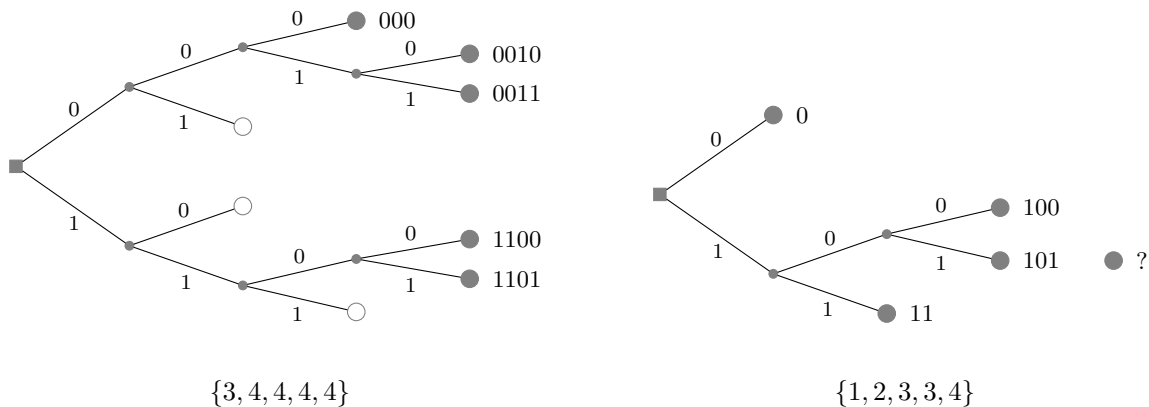


**Figure 3.2:** Binary tree representation of some source codes from Example 3.3. The white circle node indicates a leaf with no codeword.

Therefore, there exists a prefix-free codes with such lengths, for example  $\{000, 0010, 0011, 1100, 1101\}$ . Instead, for a length distribution  $\{1, 2, 3, 3, 4\}$ , we cannot construct a prefix-free code since

$$2^{-1} + 2^{-2} + 2 \cdot 2^{-3} + 2^{-4} = \frac{17}{16} \geq 1 \quad (3.7)$$

A possible attempt at constructing such a code is shown in Figure 3.3, where there is one codeword of length 4 that cannot be connected anywhere. It is not possible to construct such a code.



**Figure 3.3:** Example of a binary tree representation of the source codes in Example 3.7.

We end this lecture by stating a fundamental result in data compression.

**Theorem 3.8.** The average length of any prefix-free code of lengths  $\{\ell_1, \dots, \ell_r\}$  used to represent a source  $U$  taking values in  $\{1, \dots, r\}$  with probability distribution  $\{p_1, \dots, p_r\}$  satisfies

$$L_{\text{av}} \geq H(U), \quad (3.8)$$

where  $H(U)$  is **the entropy of the source**, measured in bits, and is given by

$$H(U) = - \sum_{j=1}^r p_j \log_2 p_j. \quad (3.9)$$

Note that in order to meet inequality (3.8) with equality, the lengths should be such that  $2^{-\ell_j} = p_j$  exactly for all  $j = 1, \dots, r$ .

**Problem 3.9.** Consider a source  $U$  taking values in  $\{1, 2, 3\}$  with equal probability. Find a prefix-free code of length distribution  $\{1, 2, 2\}$  and check that Theorems 3.6 and 3.8 are satisfied.

*Solution:* A possible prefix-free code with such lengths is given by  $\{0, 10, 11\}$ . The code is uniquely decodable by Theorem 3.4, it satisfies Kraft's inequality in Theorem 3.6 with equality since  $\sum_{j=1}^r 2^{-\ell_j} = 1$ . This means that the binary tree associated with  $\{0, 10, 11\}$  has no empty leaves! When used to represent the source  $U$ , it has average length  $L_{\text{av}} = \sum_{j=1}^r p_j \ell_j = \frac{5}{3} \approx 1.667$  bits per symbol, and is bounded by the entropy of the source from below since

$$H(U) = - \sum_{j=1}^r p_j \log_2 p_j = \log_2 3 \approx 1.585. \quad (3.10)$$

The result in (3.8) states that the entropy  $H(U)$  is the minimum average length required to represent a source  $U$ , we cannot represent it without errors with less bits per symbol. From all possible prefix-free codes, we wish to find the one that minimizes the average length in (3.4). This will be the topic of our next lecture.